DQN: Playing Atari with Deep Reinforcement

[개념 정리]

파이썬과 케라스로 배우는 강화학습

1. 강화학습 개요 (p.4~18)
2. 강화학습 기초 1: MDP와 벨만 방정식 (p.28-60)
3. 강화학습 기초 2: 그리드월드와 다이내믹 프로그래밍 (p.67-75, p.92-96)

📒 강화학습 개요

1. **강화학습의 개념**

- 시행착오를 통해 학습하는 방법
- 머신러닝: 데이터를 학습해서 스스로 성능을 높여가는 것 (지도학습 / 비지도학습 / 강화학습)
    - **지도학습**: 정답을 알고있는 데이터로 컴퓨터학습 (ex. 회귀분석, 분류, 딥러닝의 MNIST)
    - **비지도학습:** 정답 없이 주어진 데이터로만 학습 (ex. 군집화)
    - **강화학습:** "보상"을 통해 학습. 보상(컴퓨터가 선택한 행동에 대한 환경의 반응)은 간접적인 정답의 역할을 함. —> 보상을 얻게 하는 행동을 많이하도록 학습

2. **스스로 학습하는 컴퓨터, 에이전트**

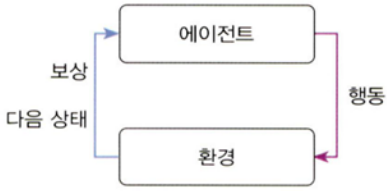


그림 1.4 에이전트와 환경의 상호작용

- 에이전트: 강화학습으로 스스로 학습하는 컴퓨터
    - 환경 사전지식X 시작 → 환경에서 자신의 상태 인식 후 행동 → 환경은 에이전트에게 보상, 다음 상태 알려줌 → 반복 → 보상을 지속적으로 얻으며 좋은 행동 학습

- 에이전트가 최적의 행동양식 및 정책을 학습하는 것이 목표
- 상벌을 적절히 융합해 효과적 학습 (보상이 음수면 처벌)
- 장점: 환경에 대한 사전지식 없이 학습가능. 시행착오를 통해 기능을 학습

- 알파고도 바둑 모른 상태로 학습시작, 이기는 행동을 학습한 것.

3. **강화학습 문제**

A. 순차적 행동 결정 문제

- 결정을 순차적으로 내려야하는 문제
- 다이내믹 프로그래밍, 진화 알고리즘도 가능하나 강화학습이 한계를 극복가능
- MDP: 순차적 행동결정 문제를 수학적으로 정의하는 구성요소들
- 구성요소
  - **상태:** 현재 에이전트의 정보 (정적/동적) - ex. 탁구공의 위치, 속도, 가속도
  - **행동:** 에이전트가 상태에서 취할 수 있는 행동. - ex. 게임에서의 입력
    - 학습하면서 특정 행동 확률을 높임
  - **보상:** 에이전트가 학습할 수 있는 정보. 시간에 따라 얻는 보상들의 합을 최대로 하는 정책을 찾음.
  - **정책:** 구해야하는 답. 모든 상태에 대해 에이전트가 어떤 행동을 해야하는지 정해놓은 것. 제일 좋은 최적 정책에 따라 행동하면 보상의 합을 최대로 얻음

B. 방대한 상태를 가진 문제

- 알파고
- 로봇의 학습 문제 - 로봇의 정보, 행동, 보상이 모두 연속적이므로 경우의 수가 무한대임

⇒ 수많은 정보를 함수 형태로 근사하는 인공신경망을 사용해야 함

📒 강화학습 기초1: MDP와 벨만 방정식

1. **MDP**

- 순차적으로 결정해야하는 문제를 순차적으로 표현, 문제에 대한 “정의”

- MDP의 구성요소: 상태, 행동, 보상함수, 상태변환 확률, 할인률
- **상태 S**: 에이전트가 관찰 가능한 상태의 집합 → 사용자가 정의해야함.
  - 그리드월드 상태 집합: $S = (1,1),(1,2),(1,3),\ldots,(5,5)$
  - 시간 t에 (1,3) 상태 ⇒ $S_t = (1,3)$
  - 상태는 시간에 따라 확률적으로 변함. ⇒ 확률 변수

$$\boldsymbol{S} = \{(x_1,y_1),(x_2,y_2),(x_3,y_3),(x_4,y_4),(x_5,y_5)\}$$

수식 2.1 상태의 집합

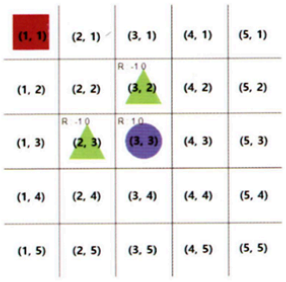


그림 2.2 그리드월드에서 상태는 좌표를 의미한다

- **행동 A:** 에이전트가 S_t에서 할 수 있는 가능한 행동의 집합
  - 보통 모든 상태에서 같음.
  - 시간 t에서의 행동 a ⇒ $A_t = a$
  - 그리드월드 행동 집합: A = {up, down, left, right}
  - 상태 변환 확률 계산 후 이동 결정
- **보상함수:** 에이전트가 학습할 수 있는 유일한 정보로 환경이 에이전트에게 주는 정보.
  - 상태 S_t = s이고 행동 A_t = a일때 보상에 대한 기댓값 E

$$r(s,a) = E[R_{t+1} \mid S_t = s, A_t = a]$$

수식 2.7 보상함수의 정의

    - 환경이 에이전트에게 보상을 알려주는 것은 t+1 시점이므로 보상은 R_{t+1}
    - 하나의 시간 단위가 지난 다음 보상 받음 → "타임스텝"
- **상태 변환확률 P**: 상태의 변화에는 확률적인 요인이 들어감

$$P_{ss'}^{a} = P[S_{t+1} = s' \mid S_t = s, A_t = a]$$

수식 2.12 상태 변환 확률

  - 상태 s에서 행동 a를 했을때 다른 상태 s에 도달할 확률
  - 환경의 일부(에이전트가 모르는 값)
  - 상태 → 에이전트 행동 → 환경이 상태변환 확률로 다음 상태 알려줌

- **할인률:** 보상이 얼마나 시간이 지나서 받는지를 고려함
  - 현재에 가까운 보상일수록 더 큰 가치
  - 미래의 가치를 현재의 가치로 환산하는 것을 할인으로 표현

$$\gamma^{k-1} R_{t+k}$$

수식 2.14 할인율을 고려한 미래 보상의 현재 가치

- **정책:** 모든 상태에서 에이전트가 할 행동
  - 입력(상태) → 출력(행동)
  - 최적 정책을 학습해야함, 확률적으로 여러개의 행동을 선택해야 다양한 상황을 학습하고 최적 정책을 찾을 수 있음

$$\pi(a \mid s) = P[A_t = a \mid S_t = s]$$

수식 2.15 정책의 정의

  - 현재 정책보다 더 좋은 정책을 계속 학습

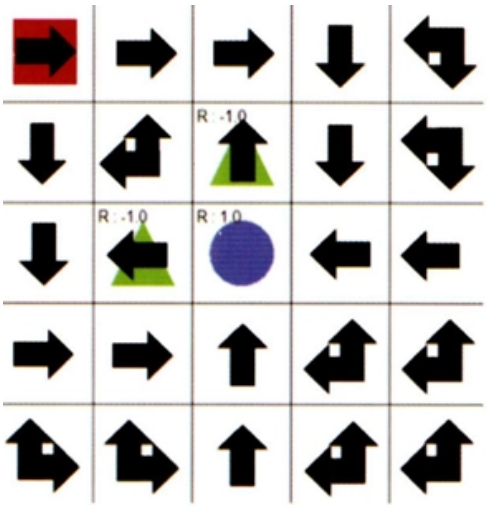


그림 2.8 그리드월드에서 정책의 예시

2. 가치함수

- 앞으로 받을 보상. 가치함수로 행동을 선택함.
  - 행동을 하면 다음 타임스텝에서 보상을 받음 (R_{t+1}). 현재 t로부터 에이전트가 행동을 하면서 받을 보상들의 합=$R_{t+1} + R_{t+2} + R_{t+3} \ldots$
    - R은 확률 변수
    - 단순합은 보상의 시점을 고려하지 않고, 한번에 받는 보상과 여러 번 나눠받는 보상을 구분할 수 없고, 시간이 무한대일경우 구분할 수 없는 문제
    - "할인률" 개념 사용 ⇒ "반환값"
  - 반환값: 에이전트가 실제로 환경을 탐험하며 받은 보상의 합.
    - 에이전트는 환경과 유한한 시간동안 상호작용 후 마지막 상태에서 반환값(보상)을 계산함.
    - 상태의 가치를 판단하기 위해 반환값의 기댓값을 계산함 (상태 가치가 가장 높은 상태를 선택해야하기 때문)

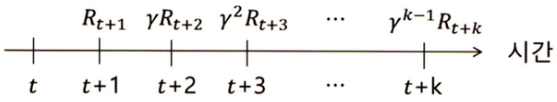


그림 2.13 할인율을 고려한 미래 보상의 현재 가치

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

수식 2.19 반환값의 정의

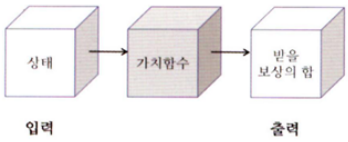
- **(상태)가치함수:** $v(s) = E[G_t | S_t = s]$
  - 상태의 가치. 보상 반환값의 합의 기댓값. 확률 변수가 아닌 특정 양을 나타내는 값
  - 앞으로 받을 보상에 대한 기댓값의 비교로 선택이 가능함
  - 항상 정책에 따라 계산해야함
  - 벨만 기대 방정식(2.25): 현재 상태의 가치함수 $v(s)$와 다음상태의 가치함수 $v(S_{t+1})$ 사이의 관계 설명

$$v(s) = E[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} \cdots) \mid S_t = s]$$
$$v(s) = E[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

수식 2.23 반환값으로 나타내는 가치함수

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 2.25 정책을 고려한 가치함수의 표현



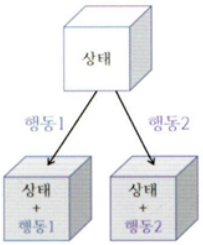
- **큐 함수 = 행동 가치함수** $q(s, a)$
  - 각 행동에 대해 가치를 알려주는 함수 → 행동의 선택
  - 보통 행동 선택 기준으로 가치함수보다 큐함수를 사용함
  - 벨만 기대 방정식(2.27)으로 표현가능 (조건문에 행동이 들어감)

$$q_\pi(s,a) = E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

수식 2.27 큐함수의 정의

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s) q_\pi(s,a)$$

수식 2.26 가치함수와 큐함수 사이의 관계식



3. 벨만 방정식

1. 벨만 기대 방정식

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 2.31 벨만 기대 방정식

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)\left(r_{(s,a)} + \gamma \sum_{s' \in S} P^a_{ss'} v_\pi(s')\right)$$

수식 2.32 계산 가능한 벨만 방정식

- 현재 상태의 가치함수와 다음상태의 가치함수 사이의 관계를 식으로 나타낸 것
- 앞으로 받을 모든 보상을 한번에 계산하는 것이 아닌, 값을 변수에 저장하고 루프를 도는 계산을 통해 참 값을 알아나가는 방식
- 기댓값 → 어떤 행동을 할 확률 π, 행동 시 어떤 상태로 가는 확률 P 포함해 계산
- 예)
  - 현재의 가치함수 0, 각 행동은 25% 확률로 선택, 상태의 가치함수는 V
  - (1) 각 행동을 할 확률 (2) 각 행동 시 보상 (3) 다음 상태의 가치함수 ⇒ (표 2.1) 현재의 가치함수 업데이트
  - 계속 업데이트 시 실제 보상의 값에 대한 참 기댓값을 얻을 수 있음

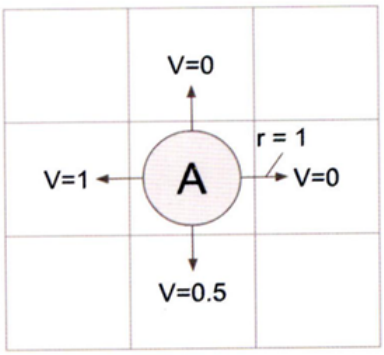


그림 2.17 그리드월드에서 가치함수의 업데이트

표 2.1 벨만 기대 방정식의 계산

1	행동 = 상	$0.25 \times (0 + 0.9 \times 0) = 0$
2	행동 = 하	$0.25 \times (0 + 0.9 \times 0.5) = 0.1125$
3	행동 = 좌	$0.25 \times (0 + 0.9 \times 1) = 0.225$
4	행동 = 우	$0.25 \times (1 + 0.9 \times 0) = 0.25$
총합	기댓값 $= 0 + 0.1125 + 0.225 + 0.25 = 0.5875$	

2. **벨만 최적 방정식**

- 참 가치함수 ≠ 최적 가치함수
  - 참 가치함수: 어떤 정책을 따라 움직였을 때 받게 되는 보상에 대한 참 값
  - 최적가치함수: 수많은 정책 중 가장 높은 보상을 얻게 되는 정책을 따랐을 때 가치함수

- 벨만 기대 방정식으로 현 정책에 대한 참 가치함수를 구할 수 있음. 강화학습은 MDP로 정의되는 문제에서 최적 정책을 찾는 것이므로 가장 큰 가치함수를 가진 행동을 하는 정책을 찾음 (max)

- 벨만 최적 방정식:
  최적의 가치함수(최적의 큐함수 중 max)를 찾는 방정식
  - 선택 상황에서 판단 기준은 !큐함수!
  - 최적 정책을 따라갈 때 현재 상태의 큐함수는 다음 상태의 행동 중 가장 높은 값의 큐함수를 1번 할인하고 보상을 더한 것

$$v_*(s) = \max_a [q_*(s,a) \mid S_t = s, A_t = a]$$

수식 2.39 큐함수 중 최대를 선택하는 최적 가치함수

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 2.40 벨만 최적 방정식

$$q_*(s,a) = E[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a]$$

수식 2.41 큐함수에 대한 벨만 최적 방정식

📒 강화학습 기초 2: 그리드월드와 다이내믹 프로그래밍

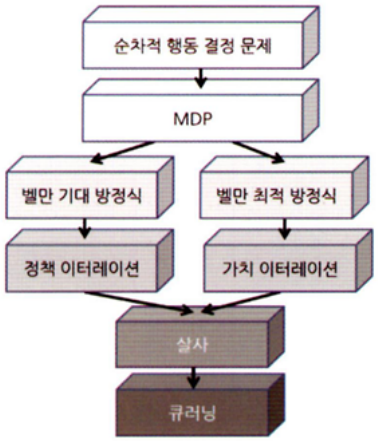


그림 3.4 강화학습 알고리즘의 흐름도

강화학습 기본 알고리즘

- MDP로 문제를 수학적으로 정의 → 목표는 에이전트가 받을 보상의 합의 최대 → 가치함수로 보상의 합의 기댓값을 계산 (벨만 방정식 = 벨만 기대방정식 / 벨만 최적 방정식)
- 벨만 방정식 →"다이나믹 프로그래밍"으로 풀 수 있음
  - 정책 이터레이션
  - 가치 이터레이션

1. 정책 이터레이션

1.1 개념

- 벨만"기대"방정식을 사용해 MDP로 정의되는 문제를 푸는 다이내믹 프로그래밍
- 무작위로 행동을 정하는 정책으로 시작 → 계속 발전시키는 방법: **"정책 평가" ⇒ "정책 발전"**
  무한히 반복하면 최적 정책으로 수렴

1.2 정책 평가

- 정책 판단의 근거는 "가치함수"(현재 정책 시 보상 기댓값)

- 계산량 유한하게 하기 위해 다이내믹 프로그래밍 수행 → **벨만 기대 방정식** 사용
  - 주변 상태의 가치함수와 한 타임스텝의 보상만 고려해 현재 다음 가치함수를 계산
  - 이 계산을 여러번 하면 참 값으로 수렴 ; π정책에 대해 반복적 수행
  - 기댓값(확률적 부분)을 합의 형태로 변형하여 전개

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 3.4 벨만 기대 방정식을 통한 효율적인 가치함수의 계산

$$v_{k+1}(s) = \sum_{a \in A} \pi(a \mid s)(r_{(s,a)} + \gamma v_k(s'))$$

수식 3.6 k번째 가치함수를 통해 k+1번째 가치함수를 계산한다

- ex) 벨만 기대 방정식의 계산방법
  - 정책의 상,하,좌,우 선택 확률 0.25 / 상태변환확률 1 / 할인율 0.9
  - 가치함수 업데이트 위해서는 다음 타임스텝 상태의 가치함수를 이용해야함 (모든 상태에 대해 동시 진행)

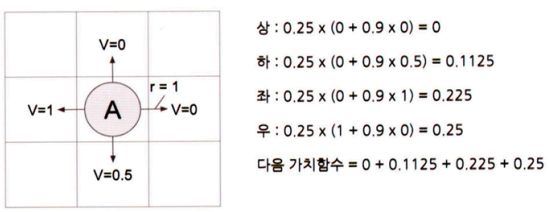


그림 3.7 그리드월드에서 가치함수의 계산

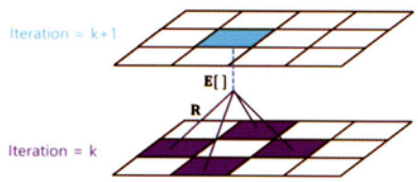


그림 3.8 벨만 기대 방정식을 이용한 현재 상태의 가치함수 업데이트

- 정책 평가 과정 x 1 :
  1. k번째 가치함수 행렬에서 현재 다음 상태 s′에 저장된 가치함수$(v_k(s'))$ 불러옴
  2. $v_k(s')$ * 할인율 γ + 보상 $r_{(s,a)}$
  3. 2번 값 * 정책값(그 행동을 할 확률) $\pi(a|s)$
  4. 모든 선택 가능한 행동에 대해 3번 반복, 다 더함 $\sum$
  5. 4번 값을 k+1번째 가치함수 행렬의 상태 s자리에 저장
  6. 1~5 모든 s에 대해 반복

  → 이 정책 평가 과정을 여러번 반복 ⇒ $v_1 \rightarrow v_\pi$ 참 값에 수렴

1.3 정책 발전

- 정책 평가 후 각 행동의 가치를 알 수 있음

- 정책 평가에서 모든 상태에 대한 가치를 구함 → 큐함수로 각 상태에 대해 어떤 행동이 좋은지 알 수 있음

- 탐욕 정책 발전: 상태s에서 선택가능한 행동의 큐함수 비교 후, 가장 큰 큐함수를 가지는 행동을 선택함. (당장 가장 큰 이익 추구)
  - 이전 가치함수에 비해 업데이트된 정책으로 받을 가치함수가 무조건 크거나 같음. 단기적으로 무조건 이익
  - 장기적으로는 가장 큰 값의 가치함수를 가지는 최적 정책에 수렴가능

$$q_\pi(s,a) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 3.8 큐함수의 정의

$$q_\pi(s,a) = r_{(s,\,a)} + \gamma v_\pi(s')$$

수식 3.9 계산 가능한 형태로 고친 큐함수

$$\pi'(s) = \mathrm{argmax}_{a \in A}\, q_\pi(s,a)$$

수식 3.10 탐욕 정책 발전으로 얻은 새로운 정책

2. 가치 이터레이션

2.1 명시적인 정책과 내재적인 정책

- 정책 이터레이션 (명시적 정책):
  - 정책은 확률적임
  - 정책과 가치함수가 독립적임, 확률적 정책이므로 기댓값 사용 → 벨만 기대방정식 사용
- 가치 인터레이션 (내재적 정책)
  - 정책을 확률 대신 결정적 형태로 가정
  - 정책이 가치함수 안에 내재적으로 포함 → 가치함수 업데이트하면 정책도 발전됨

2.2 벨만 최적 방정식과 가치 이터레이션

- 벨만 기대 방정식 → 답은 참 가치함수
  - 현재 정책 가치함수를 반복적 계산하면 참 가치함수가 된다.
- 벨만 최적 방정식 → 답은 최적 가치함수
  - 최적 정책 가치함수를 반복적 계산하면 최적 정책에 대한 참 가치함수, 즉 최적 가치함수가 된다.
  - 따로 정책발전 필요X
  - max()를 사용하여 최고값으로만 업데이트. 정책의 값은 고려X ⇒ 가치 이터레이션

- 정책 이터레이션은 상태들을 다 고려해 업데이트 했으나, 가치 이터레이션은 제일 높은 값을 가지는 값으로만 업데이트

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 3.13 벨만 최적 방정식을 통해 구하는 것은 최적 가치함수다

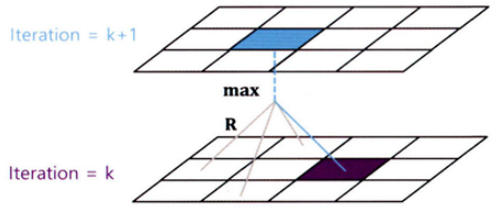


그림 3.18 $R_{t+1} + \gamma v_k(S_{t+1})$ 값 중에서 max 값으로 다음 가치함수를 계산

[논문 리뷰] DQN

📗 논문 링크: Playing Atari with Deep Reinforcement Learning

1. Introduction

논문이 다루는 분야

강화 학습(Reinforcement Learning)을 사용하여 시각과 같은 고차원 감각 입력으로부터 에이전트의 제어 정책을 직접 학습시키는 분야를 다루는 논문. 특히, raw pixels 데이터만을 입력받아 게임을 플레이하는 법을 스스로 터득하는 심층 신경망 모델을 제시하고 있음.

해당 task에서 기존 연구 한계점

기존의 강화 학습 응용 사례들은,

- 대부분의 성공적인 시스템은 사람이 직접 제작한 특징(hand-crafted features)과 선형 가치 함수에 의존하여 시스템의 성능이 특징 표현의 품질에 크게 좌우됨
- 강화 학습은 지도 학습과 달리 보상 신호가 희소하고 노이즈가 섞여 있으며, 행동과 실제 보상 사이에 수천 번의 타임스텝이 존재하는 '지연된 보상' 문제로 인해 학습이 어려웠음
- 대부분의 딥러닝 알고리즘은 샘플 간의 독립성을 가정하지만, 강화 학습 데이터는 연속적인 상태 간의 상관관계가 매우 높음
- 에이전트가 새로운 행동을 배울 때마다 데이터의 분포가 계속 변하는 **비정상성(non-stationary)** 문제로 인해 학습이 불안정하거나 발산(diverge)할 위험이 큼

논문의 contributions

- **DQN(Deep Q-Network) 제안:** 고차원 시각 데이터에서 미래 보상을 추정하는 가치 함수를 직접 학습하는 합성곱 신경망(CNN) 기반의 첫 번째 딥러닝 모델을 제시

- **Experience Replay 도입:** 에이전트의 과거 경험을 메모리에 저장한 후 무작위로 샘플링하여 학습에 사용하는 방식 → 데이터 간의 상관관계를 끊고 훈련 분포를 매끄럽게 만들어 학습의 안정성을 높임
- **범용적 학습 성능:** 별도의 아키텍처나 하이퍼파라미터 조정 없이 단일 모델로 7개의 아타리 게임 중 6개에서 기존 알고리즘을 능가하는 성능을 보임
- **엔드 투 엔드(End-to-End) 학습:** 게임별 사전 지식 없이 원시 비디오 데이터, 보상 신호, 행동 세트만으로도 복잡한 정책을 학습할 수 있음

2. Related Work

- **TD-Gammon:** 강화학습의 가장 잘 알려진 성공사례로, 강화학습과 셀프 플레이를 통해 슈퍼 휴먼 수준에 도달한 backgammon playing program
  - Q-learning과 비슷한 강화학습 알고리즘 학습
  - 주사위 굴리기의 무작위성 덕분에 상태 공간 탐색이 용이하고 가치함수가 매끄러워진것.
  - 체스, 바둑, 체커 등에 동일한 방법 적용 실패

  +. Q-learning 같은 model-free 강화학습 알고리즘 + 비선형 함수 근사기 or off-policy 학습 ⇒ 가치 함수가 발산(diverge)할 수 있음

  → 이후 대부분의 연구는 선형 근사기에 집중 (수렴 보장)

  ▼ 가치함수의 발산

  강화 학습에서 가치 함수가 발산한다는 것은 학습 과정에서 예측된 Q 가치가 정답에 수렴하지 않고, 값이 통제 불능으로 커지거나 불안정해져서 학습이 실패하는 상태를 의미함

  - **피드백 루프:** on-policy 학습 시 현재의 파라미터가 다음 훈련 데이터를 결정하게 되는데, 이 과정에서 원치 않는 피드백 루프가 생겨 파라미터가 나쁜 지역 최솟값에 갇히거나 diverge catastrophically할 수 있다고 설명
- Restriced Boltzmann Machines(RBM): 가치 함수를 추정하기 위해 딥러닝 사용 / gradient temporal difference로 발산 문제 부분 해결 → 비선형 제어 확장X
- **NFQ (Neural Fitted Q-learning):** 본 논문과 가장 유사한 선행연구로 RPROP 알고리즘을 사용해 Q-network 파라미터 업데이트
  - 배치 업데이트 사용 → 데이터 세트의 크기에 비례하는 비용

    → 본 논문에서는 확률적 경사 업데이트를 이용 (대규모 데이터 세트에 확장 가능한 낮은 비용 가능)

    ▼ 배치 업데이트 vs 확률적 경사 업데이트

    - **Batch Update:** 한 번의 파라미터 업데이트를 위해 **전체 데이터 세트**를 모두 사용. 데이터가 많아질수록 무거워짐

- **Stochastic Gradient Update:** 전체 데이터가 아닌, 무작위로 추출된 샘플(본 논문에서는 미니배치)을 사용하여 가중치를 업데이트. 반복당 계산 비용이 낮고 일정하게 유지되어 대규모 데이터 세트에서도 효율적으로 확장가능

- 딥 오토인코더를 먼저 학습시켜 저차원 표현을 얻은 후 적용

  → 본 논문에서는 시각 입력으로부터 직접 ent-to-end 강화학습 적용

3. 제안 방법론

Main Idea

✅ 기존 **표준 Q-learning**

1. Future Discounted Return 미래 감쇠 수익

   - 에이전트의 목표는 환경과 상호작용하며 미래 보상 최대화하는 행동 선택하는 것
   - 미래 보상은 타임스텝 당 discount factor γ에 의해 감쇠한다고 가정

$$R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$$

   - 미래 감쇠 수익 R_t

2. 최적 행동 가치 함수 ($Q*(s,a)$)

   어떤 시퀀스 s를 본 후 행동 a를 취했을 때, 어떤 전략을 따르더라도 얻을 수 있는 **최대 기대 수익**

3. 벨만 방정식

   현재 상태의 가치가 현재 받을 보상과 다음 상태에서 얻을 수 있는 최적 가치의 합과 같다. 최적 행동 가치 함수는 이 항등식을 따름
   → 강화학습 알고리즘에서는 반복적 업데이트로 Q 함수 추정(무한으로 반복 시 최적 함수 Q∗에 수렴)

$$Q^*(s,a) = \mathbb{E}_{s'\sim E}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s,a\right]$$

$$Q_{i+1}(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q_i(s',a') \mid s,a\right]$$

4. 표준 Q learning

   위 식의 기대값을 계산하는 대신 실제 환경에서 얻은 단일 샘플을 사용해 가중치를 업데이트하면 Q learning 알고리즘이 됨

   - Model free: 환경에 대한 명시적 모델 구축 없이 샘플 직접 사용
   - Off policy: 상태 공간을 충분히 탐험하기 위한 행동 분포를 따르면서도 최적의 그리디 전략 학습
   - 각 시퀀스(상태)에 대해 별도로 가치를 추정하므로 일반화가 불가능
     → 본 논문에서는 신경망을 이용한 함수 근사기 $Q(s,a;\theta) = Q*(s,a)$도입

✅ 심층 강화 학습 (Deep Reinforcement Learning) ✔️

- **목표:** 강화 학습 알고리즘 + RGB 이미지 적용가능한 심층 신경망 + 확률적 경사 업데이트 사용 → 효율적 학습
- **경험 재플레이(experience replay) 기술**
  - 에이전트의 경험 $e_t = (s_t, a_t, r_t, s_{t+1})$을 데이터 세트 *D*에 저장 → 무작위 샘플링된 경험들에 대해 Q-학습 업데이트(또는 미니배치 업데이트) → 에이전트는 ϵ-그리디 정책 따라 행동 선택, 실행
  - 신경망 입력으로 함수ϕ 에 의해 생성된 고정된 길이의 히스토리 사용
  - 전체 알고리즘: 경험 재플레이를 이용한 Deep Q-learning
    1. **준비**
       a. 재생 메모리 *D*를 용량 *N*으로 초기화
       : AI가 겪을 일을 저장할 재생 메모리 D 준비
       b. 무작위 가중치 *θ*로 행동 가치 함수 Q 초기화
       : 게임 상황을 판단할 신경망(Q함수) 초기화
    2. **게임 플레이 및 저장** (각 (게임)에피소드 1부터 M까지 반복):
       a. 시퀀스 $s_1 = x_1$ 및 전처리된 시퀀스 $\phi_1 = \phi(s_1)$ 초기화
       b. 각 타임스텝 *t*=1부터 *T* 반복:
       - 행동선택: ϵ 확률로 무작위 행동 a_t선택, 그렇지 않으면 $a_t = max_{a'} Q * (\phi(s_t), a; \theta)$ 선택 → 가끔 무작위 새로운 시도, 보통은 지금까지 배운 것 중 가장 점수가 높을 것 같은 행동 선택
       - 경험 쌓기: 행동 실행 뒤, 그 결과로 얻은 화면, 보상, 다음 화면을 D에 저장함 = <u>Transition</u>
         - a_t실행 후 보상 r_t와 이미지 x_{t+1} 관찰
         - $s_{t+1} = s_t, a_t, x_{t+1}$설정 및 $\phi_{t+1} = \phi(s_{t+1})$ 전처리
         - Transition $(\phi_t, a_t, r_t, \phi_{t+1})$를 *D*에 저장
       - **경험 재플레이** :복습하며 학습- 바로 직전의 경험 뿐 아니라 무작위로 예전 경험을 꺼내어 학습
         - D 에서 <u>무작위 미니배치 Transition $(\phi_j, a_j, r_j, \phi_{j+1})$ 샘플링</u>
         - 타겟 y_j설정: 종료 상태면 r_j, 아니면 $r_j + \gamma max_{a'} Q(\phi_{j+1}, a'; \theta)$
           → 정답지 만들기(그때 그 상황(ϕ_j)에서 이 행동(a_j)을 했다면, 실제로 받은 점수와

그다음 상황에서 얻을 수 있는 최대 점수의 합은 이 정도

- 오답 업데이트: $(y_j - Q(\phi_j, a_j; \theta))^2$에 대해 경사 하강법 단계 수행
  → AI가 현재 추측하는 점수와 방금 만든 정답지의 차이를 계산, 차이가 줄어드는 방향으로 신경망 가중치를 업데이트

- **경험 재플레이의 장점 (vs 표준 Q-learning)**
  - **데이터 효율성:** 한 번 겪은 경험을 버리지 않고 여러 번 학습에 활용할 수 있어 효율적, 각 경험 단계가 잠재적으로 많은 가중치 업데이트에 사용됨
  - **상관관계 제거:** 게임 화면은 앞뒤가 서로 매우 비슷(강한 상관관계)해서 순서대로 배우면 특정 상황에만 편향될 수 있는데, 무작위로 섞어 배우면 이를 방지하고 학습의 분산을 줄여줌
  - **학습의 안정성 - 피드백 루프 방지:** AI가 현재 파라미터가 다음 훈련 데이터를 결정하여 파라미터가 나쁜 지역 최솟값(local minimum)에 갇히거나 학습이 발산할 수 있는 피드백 루프 현상을 방지

- 마지막 *N*개의 경험 튜플만 저장하고 업데이트 시 균등하게 무작위 샘플링
  ⇒ 메모리 크기 제한으로 인해 오래된 Transition를 덮어쓰고 모든 Transition에 동일한 중요도를 부여한다는 한계가 있지만 매우 효과적임

4. 실험 및 결과

Dataset

1. **사용 데이터**

   아케이드 학습 환경(ALE, Arcade Learning Environment)을 통해 제공되는 Atari 2600 게임의 원시 비디오 데이터(Raw Video Data)

   - 총 7가지의 유명한 Atari 게임 데이터: **Beam Rider, Breakout, Enduro, Pong, Q*bert, Seaquest, Space Invaders**
   - 입력 데이터: 에이전트(AI)는 게임의 내부 상태가 아닌, 사람 플레이어와 동일하게 화면의 **Raw Pixels**만을 입력받음.(210 x 160 해상도의 RGB 비디오(60Hz) 형태, 128개의 색상 팔레트)
   - 추가 신호:
     - 보상(Reward) 신호: 게임 점수의 변화를 나타냄
     - 종료(Terminal) 신호: 게임의 종료 여부
     - 행동 세트(Action set): 게임에서 가능한 행동 모음

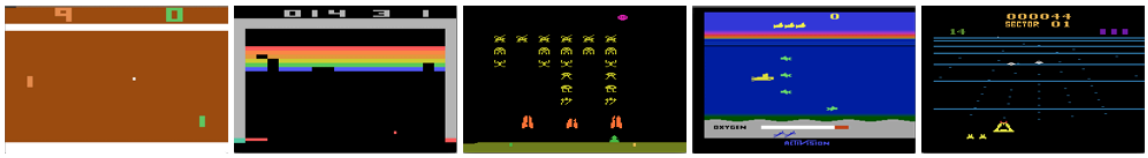


Figure 1: Screen shots from five Atari 2600 Games: (*Left-to-right*) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

2. **데이터 전처리**

   원시 Atari 화면(210×160 RGB, 128색)은 계산량이 너무 많기 때문에 차원을 줄임

   - RGB 프레임을 그레이스케일로 바꾸고 110×84로 해상도를 낮춤
   - **Cropping:** 최종적으로 게임 플레이 영역만 포함하는 84×84 크기로 크롭
     → GPU 연산 최적화를 위해 정사각형 입력을 맞춤
   - **Stacking:** 함수 ϕ를 통해 전처리된 최근 4개의 프레임을 하나로 쌓아서 신경망의 입력 (84×84×4)으로 사용

Baseline

1. **DQN 신경망 아키텍처**

   입력된 상태에서 가능한 모든 행동의 Q-가치를 한 번의 forward pass로 동시에 계산하여 효율성을 극대화

   - **입력층:** 전처리된 84×84×4 이미지
   - **은닉층 1 (Conv):** 16개의 8×8 필터 (스트라이드 4) + ReLU 활성화 함수
   - **은닉층 2 (Conv):** 32개의 4×4 필터 (스트라이드 2) + ReLU 활성화 함수
   - **은닉층 3 (Fully-connected):** 256개의 유닛 + ReLU 활성화 함수
   - **출력층:** 각 유효한 행동에 대한 Q-가치를 출력하는 완전연결 선형 레이어

2. **실험 설정 및 하이퍼파라미터**

   * 7개의 다양한 게임에 대해 동일한 아키텍처와 설정을 적용

   - **Reward Clipping:** 게임마다 점수 스케일을 통일하기 위해 모든 양의 보상은 **1**, 음의 보상은 **1**, 변동 없는 경우는 **0**으로 고정함 → 학습률을 여러 게임에 공통적으로 적용가능
   - 미니배치 크기 **32**의 **RMSProp** 알고리즘 사용
   - **탐험 전략 (ϵ그리디):** ϵ 값을 처음 100만 프레임 동안 **1에서 0.1로 서서히 낮추며(annealing)** 탐험을 유도하고, 이후에는 0.1로 유지

- **Frame-skipping:** 매 프레임마다 행동을 고르는 대신 매 4번째 프레임(k=4)마다 행동을 선택하여 연산 효율을 높임. (다만, 레이저가 깜빡여서 안 보일 수 있는 'Space Invaders' 게임만 예외적으로 k=3을 사용)

3. **훈련 및 안정성**

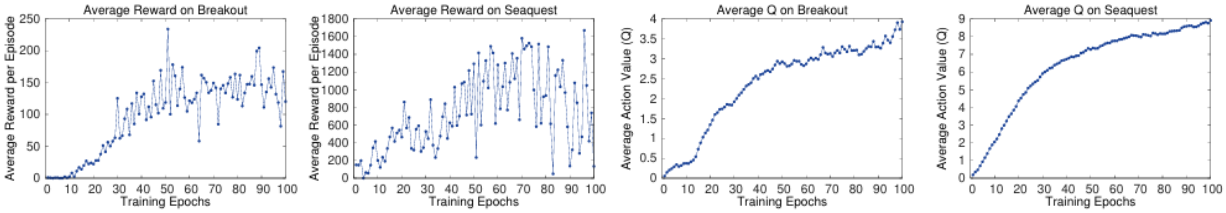
강화 학습에서는 훈련 중 에이전트의 진행 상황을 평가하기 어려움

- 평가 지표인 에피소드 당 평균 보상은 정책 가중치의 작은 변화에도 방문 상태 분포가 크게 변할 수 있어 매우 노이즈가 심하고 학습이 꾸준히 진행되는지 판단하기 어려움

  (왼쪽 두 그래프-Seaquest와 Breakout-에서 평균 보상이 매우 출렁임)

- 안정적인 지표 ⇒ 예측 Q값: 정책의 추정된 행동 가치 함수 Q를 더 안정적인 지표로 제시함. 학습 중 고정된 상태 집합에 대해 최대예측 Q값의 평균을 추적한 결과, 평균예측 Q가 매우 부드럽게 증가함

  (오른쪽 두 그래프 - 평균 예측 Q가 훨씬 부드럽게 증가, 훈련 중 발산 문제 없이 안정적으로 학습됨)



4. **가치 함수 시각화 (Visualizing the Value Function)**

- Seaquest 게임에서 학습된 가치 함수의 시각화로 AI의 상황 판단 확인
  - 적이 화면 왼쪽에서 나타나면 예측된 가치가 상승하고(지점 A), 에이전트가 어뢰를 발사하여 적을 맞추기 직전에 가치가 최고점에 도달하며(지점 B), 적이 사라지면 다시 원래 가치로 떨어짐(지점 C)
  - DQN이 복잡한 일련의 사건에 대해 가치 함수가 어떻게 진화해야하는지 잘 학습함을 보임

5. **주요 평가**

DQN의 성능을 기존 강화 학습 알고리즘(Sarsa 및 Contingency) 및 인간의 기록과 비교

→ 기존 알고리즘들은 배경 제거, 색상별 채널 분리 등 시각적 문제에 대한 사전 지식 사용
→ DQN은 원시 RGB 스크린샷만 입력받아 스스로 객체 감지

결과

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
Sarsa [3]	996	5.2	129	−19	614	665	271
Contingency [4]	1743	6	159	−17	960	723	268
DQN	**4092**	**168**	**470**	**20**	**1952**	**1705**	**581**
Human	7456	31	368	−3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	**1720**
HNeat Pixel [8]	1332	4	91	−16	1325	800	1145
DQN Best	**5184**	**225**	**661**	**21**	**4500**	**1740**	1075

Table 1: The upper table compares average total reward for various learning methods by running an ϵ-greedy policy with $\epsilon = 0.05$ for a fixed number of steps. The lower table reports results of the single best performing episode for HNeat and DQN. HNeat produces deterministic policies that always get the same score while DQN used an ϵ-greedy policy with $\epsilon = 0.05$.

- DQN은 거의 사전 지식을 사용하지 않았음에도 모든 7개 게임에서 다른 학습 방법들을 능가함
  - 특히 Breakout, Enduro, Pong에서는 전문가(인간)보다 더 높은 점수를 얻음, Beam Rider에서도 인간에 근접한 성능을 보임
  - Q*bert, Seaquest, Space Invaders는 장기적인 전략(long time scales)을 세워야 하므로 인간의 성능에 미치지 못했지만 여전히 높은 성능
- 기술적 기여
  - 강화학습을 위한 딥러닝 모델 소개, raw pixels 입력만으로 어려운 제어 정책 마스터
  - 딥러닝 + 강화학습 안정적 결합 위한 경험 재플레이 메모리(Experience Replay Memory)와 확률적 미니배치 업데이트(Stochastic Minibatch Updates)기술 도입
  - 아키텍처나 하이퍼파라미터 조정 없이 테스트된 7개 게임 중 6개에서 최첨단 결과 기록(범용성)

5. 결론 (배운점)

인공지능이 별도의 지식 없이 시각 정보만으로 복잡한 환경을 스스로 학습할 수 있음을 성공적으로 보여준 연구이다. 연속적인 데이터 간 상관관계가 높은 경우에 경험 replay 기술로 상관관계를 제거할 수 있음을 알게 되었다. 스스로 객체를 탐지하고 보상으로 상황을 판단하며 신경망까지 결합한 이 기술은 인간의 학습 능력을 거의 정확히 따라간 것 같고, 장기적인 학습까지 가능해진다면 오히려 인간보다 뛰어난 지능을 갖게 될 것 같다.